Listing 7.11 `test_actions.py` (native/semantic function definitions)

```
from nexus.nexus_base.action_manager import agent_action


@agent_action                                                  #1
def get_current_weather(location, unit="fahrenheit"):      #1
    """Get the current weather in a given location"""       #2
    return f"""
The current weather in {location} is 0 {unit}.
"""      #3


@agent_action      #4
def recommend(topic):
    """
    System:                                                      #5
        Provide a recommendation for a given {{topic}}.
        Use your best judgment to provide a recommendation.
    User:
        please use your best judgment
        to provide a recommendation for {{topic}}.           #5
    """
    pass      #6
```

#1 Applies the agent_action decorator to make a function an action
#2 Sets a descriptive comment for the function
#3 The code can be as simple or complex as needed.
#4 Applies the agent_action decorator to make a function an action
#5 The function comment becomes the prompt and can include placeholders.
#6 Semantic functions don't implement any code.

Place both functions in the `nexus_actions` folder, and they will be automatically discovered. Adding the `agent_action` decorator allows the functions to be inspected and automatically generates the OpenAI standard tool specification. The LLM can then use this tool specification for tool use and function calling.

Listing 7.12 shows the generated OpenAI tool specification for both functions, as shown previously in listing 7.11. The semantic function, which uses a prompt, also applies to the tool description. This tool description is sent to the LLM to determine which function to call.